The Brilliant Employee · 02

Your AI tool logs every session in plaintext

Mine held 2 real credentials. Anthropic documents both halves: token patterns redacted before a shared session uploads, and the copy on your own disk stored in plaintext. A request to scrub the local file was closed as not planned.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

A language model writes a large share of my production code. Wrong work reads exactly like right work, and no prompt fixes that. So the system below exists to find out when the answers were wrong.

From one week, none of it visible from inside the tool: a sizing step ignored on 796 of 816 tasks, and two fixes written up as done that were never made.

THE SYSTEM, AS OF 7 AUGUST 2026, 14:51Z

rules written	11 June, 57 days ago
ledger running	27 June, 41 days
tasks logged	3,103, across 3,620 rows
controls on tool calls	**6 registered, 4 that can refuse**

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. Each post takes one of them.

sizing check	sizes a job before a model is picked
model picker	recommends which model gets the job
safety stops	refuse the irreversible, before it runs
grader	grades finished work pass or fail
task log	at least one append-only line per task

Listed in the order they act on a task, not the order they were built.

Today cuts across more than one, so no row is marked.

> **ASIDE.** Only the safety stops can refuse. The other four observe: one advises, one recommends, one grades after the fact, one records.

This page is the map. Nothing here needs another post to make sense.

My AI auditor named a file and a credential type

An audit subagent swept a repository and came back specific and confident.

> **Plain words: PERSONAL ACCESS TOKEN.** a long random string that stands in for your password when a PROGRAM talks to a service. Anyone holding it is you. GitHub, Vercel, Supabase and Cloudflare each issue their own.

WHAT THE FINDING CARRIED	
the path named	**testing/workspaces.json**
the credential named	personal access token

A named path is checkable in seconds.
Check it before repeating anything.

The named file never existed

I checked the path before repeating the finding. There is no testing/workspaces.json. Never was.

EXIT ONE

do

close it as a hallucination

cost

you drop a real finding

EXIT TWO

do

soften it to a maybe

cost

nobody can settle it

ASIDE. Both end the search at the moment it should widen. The address was never the load-bearing part. The credential type was.

A wrong path does not refute the credential type. Those are two claims.

Search for the token shape, not the named file

1. Alarm names a specific path.
2. Check that path live. It does not exist.
3. Search the whole machine for the token PATTERN instead.

Step three is the pivot: stop auditing the claim, start auditing the machine.

```
$ find . -path '*testing/workspaces.json'
  (nothing)

$ grep -rlE 'gh[pousr]_[A-Za-z0-9]{20,}' \
    ~/.claude/projects --include '*.jsonl'
  47 files
```

One grep over the whole machine, for the pattern rather than the path.

The leak was in my chat logs

Every session my system runs is written to a log on my own disk, line by line, as it happens. Paste a secret, or let a tool echo one, and it lands there in the clear.

WHERE THE ALARM POINTED

path

testing/workspaces.json

exists

no

WHERE THE TOKENS WERE

path

~/.claude/projects/*.jsonl

exists

15,110 files

ASIDE. A request to scrub that file was filed against claude-code and closed as not planned.

Session transcripts write themselves. Nothing asks you first.

Count distinct strings, not grep hits

1. Never repeat it as fact before a live check
2. Never dismiss it on a wrong pointer. Wrong address, real fire is a common shape
3. Widen from the named location to the pattern

47 of 15,110 files — matched the pattern

163 occurrences — of 20 distinct strings

7 at 40 characters — the length a real one is

Two real credentials — what reading the 7 left

ASIDE. Re-derived 2026-08-04 22:54Z, pattern gh[pousr]_ then 20 or more characters.

163 occurrences were 20 strings were 2 real credentials.

Every vendor already told you not to do this

PRINTED ON THE PAGE THAT ISSUES THE TOKEN	
Cloudflare	shown once; never in plaintext
Supabase	never over chat or email
GitHub	never hardcode; scope it, expire it
Vercel	returned once; has a leakedAt field
your AI tool	**writes the whole chat to disk**

A session is a chat. So the tool breaks two of those rules every session, without anyone deciding to.

The advice was right. It just predates a tool that logs everything you type.

A dashboard needs a human. A cron job does not.

The choice is not token versus nicer workflow. It is interactive versus unattended.

USE THE VENDOR'S LOGIN

deploy by hand
the dashboard

your own terminal
CLI login

the vendor's CI
built-in token

YOU NEED A RAW TOKEN

a scheduled job
nobody to approve

a build elsewhere
no browser

a script an agent runs
no human in the loop

Reach for a token when nobody is there to approve. Not before.

You cannot stop a leak.
You can make it cheap.

1. Fine-grained, scoped to the one repo the job touches.
2. Shortest expiry you can live with.
3. Value in one file outside every repo, never echoed.

   Rotate on doubt: create the new one, replace it everywhere, delete the old.

One of my two real leaks WAS an environment variable, held exactly as recommended, that a tool printed. Storage is a probability. Expiry is a ceiling.

Scope and expiry beat storage, because storage fails quietly.

The fix is one line my system reloads each session

LEARNED RULE #2, STANDING RULEBOOK	
seeded	**2026-06-27**
step one	verify it live before repeating it
step two	do not dismiss it on a wrong pointer

Append-only, and read at the start of every session. The cost of the incident was paid once.

Append the rule to a file the system reads on every start. Cost paid once.

Four commands. Run them on your own machine.

Count the files that match. Count DISTINCT strings, not hits. Filter by length. Read what survives. Mine went 47 to 2.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

`sgnk.ai · in/sagnikmitra · github.com/sagnikmitra`